

# OSSS 7

Question:

Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

Solution:

```
ubuntu@ubuntu:~$ nano waitpid_monitor.c
ubuntu@ubuntu:~$ gcc waitpid_monitor.c -o waitpid_monitor
ubuntu@ubuntu:~$ ./waitpid_monitor
Parent Waiting for child...
Child Process Started
Child PID: 6740
Child Process Finished
Child PID 6740 Terminated
Exit Status: 100
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if(pid < 0)
```

```
    {
```

```
        perror("Fork Failed");
```

```
        return 1;
```

```
    }
```

```
if(pid == 0)
{
    printf("Child Process Started\n");
    printf("Child PID: %d\n", getpid());

    sleep(5);

    printf("Child Process Finished\n");

    exit(100);
}
else
{
    int status;

    printf("Parent Waiting for Child...\n");

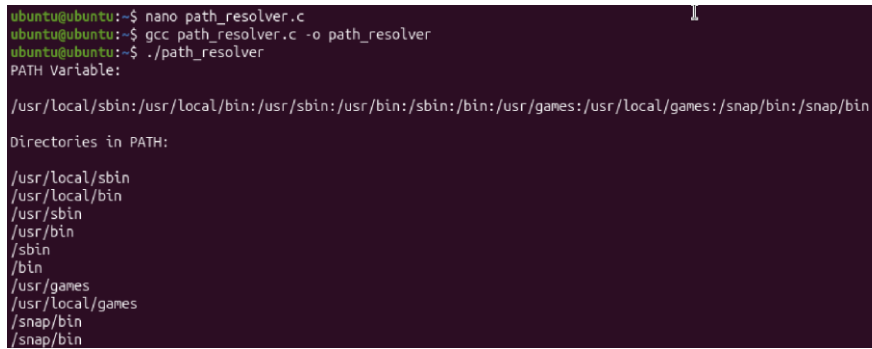
    waitpid(pid, &status, 0);

    printf("Child PID %d Terminated\n", pid);

    if(WIFEXITED(status))
    {
        printf("Exit Status: %d\n",
            WEXITSTATUS(status));
    }
}
```

```
return 0;
```

```
}
```

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~\$'. The user enters 'nano path\_resolver.c', then 'gcc path\_resolver.c -o path\_resolver', and finally './path\_resolver'. The output shows the 'PATH' variable and a list of directories in the PATH.

```
ubuntu@ubuntu:~$ nano path_resolver.c
ubuntu@ubuntu:~$ gcc path_resolver.c -o path_resolver
ubuntu@ubuntu:~$ ./path_resolver
PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin

Directories in PATH:
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    char *path = getenv("PATH");
```

```
    if(path == NULL)
```

```
    {
```

```
        printf("PATH variable not found\n");
```

```
        return 1;
```

```
    }
```

```
    printf("PATH Variable:\n\n%s\n\n", path);
```

```
    char temp[4096];
```

```
    strcpy(temp, path);
```

```
    char *dir = strtok(temp, ":");
```

```
printf("Directories in PATH:\n\n");
```

```
while(dir != NULL)
```

```
{
```

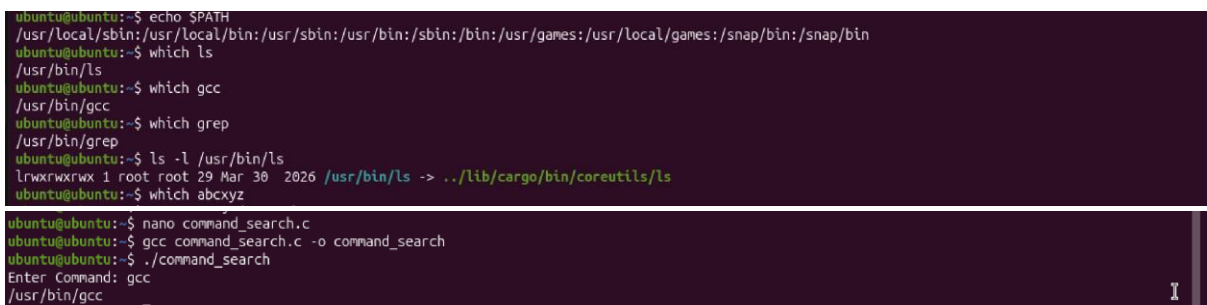
```
    printf("%s\n", dir);
```

```
    dir = strtok(NULL, ":");
```

```
}
```

```
return 0;
```

```
}
```



```
ubuntu@ubuntu:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
ubuntu@ubuntu:~$ which ls
/usr/bin/ls
ubuntu@ubuntu:~$ which gcc
/usr/bin/gcc
ubuntu@ubuntu:~$ which grep
/usr/bin/grep
ubuntu@ubuntu:~$ ls -l /usr/bin/ls
lrwxrwxrwx 1 root root 29 Mar 30 2026 /usr/bin/ls -> ../lib/cargo/bin/coreutils/ls
ubuntu@ubuntu:~$ which abcxyz
ubuntu@ubuntu:~$ nano command_search.c
ubuntu@ubuntu:~$ gcc command_search.c -o command_search
ubuntu@ubuntu:~$ ./command_search
Enter Command: gcc
/usr/bin/gcc
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    char command[100];
```

```
    printf("Enter Command: ");
```

```
    scanf("%s", command);
```

```
    char cmd[200];
```

```
sprintf(cmd, "which %s", command);
```

```
system(cmd);
```

```
return 0;
```

```
}
```